

**BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG**



ĐỒ ÁN MÔN HỌC

**MÔN HỌC: CƠ SỞ DỮ LIỆU PHÂN TÁN
ĐỀ TÀI: RECONSTRUCTION VALIDATOR:
"VEHICLE TELEMATICS"**

Giảng viên hướng dẫn: Lê Hà Thanh

Thực hiện bởi sinh viên: Đinh Ngọc Khánh Tùng

Mã số sinh viên: N23DCCN066

Lớp: D23CQCN01-N

TP.HCM, tháng 9 / 6 / 2026

MỤC LỤC

MỤC LỤC	2
CHƯƠNG I: TỔNG QUAN	3
I. Giới thiệu đề tài	3
1) Mục tiêu đề tài	3
2) Core logic	3
CHƯƠNG II: YÊU CẦU KỸ THUẬT CỦA DỮ LIỆU	4
I. Source	4
1.2. Schema	4
1.3. Fragmentation Strategy	5
1.3.1. Horizontal Fragmentation	5
1.3.2. Vertical Fragmentation	5
CHƯƠNG III: KIẾN TRÚC HỆ THỐNG	6
I. Nodes	6
II. Communication Layer	6
III. Storage	6
CHƯƠNG IV: NỀN TẢNG CÔNG NGHỆ VÀ KẾ HOẠCH THỰC HIỆN	7
I. Cấu trúc project dự kiến	7
II. Các script chính	8
CHƯƠNG V: TIÊU CHÍ THÀNH CÔNG VÀ ĐÁNH GIÁ	8
I. Quantitative Metrics	8
II. Failure Scenario	8
CHƯƠNG VI: KẾT LUẬN	9

CHƯƠNG I: TỔNG QUAN

I. Giới thiệu đề tài

Trong cơ sở dữ liệu phân tán, dữ liệu thường không được lưu tập trung tại một nơi duy nhất mà được chia thành nhiều fragment và phân bố trên nhiều site hoặc node khác nhau. Việc phân mảnh giúp tăng khả năng quản lý, mở rộng và xử lý dữ liệu, nhưng đồng thời đặt ra yêu cầu quan trọng: dữ liệu gốc phải có thể được khôi phục lại chính xác từ các fragment.

Đề tài này giải quyết bài toán kiểm tra khả năng khôi phục dữ liệu sau khi một tập dữ liệu log xe được chia thành các fragment ngang và fragment dọc. Hệ thống sẽ mô phỏng lỗi bằng cách xóa 5 dòng ngẫu nhiên từ một fragment, sau đó chạy chương trình reconstruction validator để phát hiện chính xác những bản ghi bị thiếu hoặc bị sai.

1) Mục tiêu đề tài

1. Tạo hoặc sử dụng một dataset VehicleLogs.csv gồm 10.000 dòng dữ liệu telematics của xe.
2. Thực hiện phân mảnh ngang dataset thành 4 fragment dựa trên khoảng giá trị VIN.
3. Thực hiện phân mảnh dọc mỗi fragment ngang thành 2 nhóm dữ liệu: Operational và Diagnostic.
4. Xây dựng script khôi phục dữ liệu bằng cách join các fragment dọc và union các fragment ngang.
5. Mô phỏng lỗi bằng cách xóa 5 dòng ngẫu nhiên trong một fragment.
6. Xây dựng validator để phát hiện chính xác các bản ghi bị thiếu hoặc bị sai.
7. Chứng minh các yêu cầu lossless join, lossless union, completeness, disjointness và reconstruction trong phân mảnh dữ liệu.

2) Core logic

Core logic của hệ thống gồm ba giai đoạn chính: phân mảnh dữ liệu, khôi phục dữ liệu và kiểm tra dữ liệu. Đầu tiên, dataset gốc được chia theo chiều ngang dựa trên khoảng VIN. Sau đó, mỗi fragment ngang được chia tiếp theo chiều dọc thành Operational Data và Diagnostic Data. Cuối cùng, hệ thống tái dựng dữ liệu bằng phép join trên khóa ghép VIN + Timestamp và phép union giữa các fragment ngang.

Giai đoạn	Mô tả xử lý
Fragmentation	Chia dataset gốc thành 4 horizontal fragments và 8 vertical sub-fragments.
Reconstruction	Join Operational với Diagnostic theo khóa VIN + Timestamp, sau đó union 4 fragment đã khôi phục.
Validation	So sánh dữ liệu reconstructed với dữ liệu gốc để phát hiện missing records, corrupted records và duplicate records.

CHƯƠNG II: YÊU CẦU KỸ THUẬT CỦA DỮ LIỆU

I. Source

Dataset sử dụng trong đề tài là dữ liệu giả lập được tạo bằng chương trình Python. Việc sử dụng synthetic dataset giúp nhóm kiểm soát số lượng dòng, cấu trúc dữ liệu và tái hiện chính xác các kịch bản lỗi khi demo.

Tên dataset	Nguồn dữ liệu	Định dạng	Kích thước
VehicleLogs.csv	Programmatically generated synthetic dataset	CSV	10.000 rows, khoảng 1-3 MB

1.2. Schema

Thuộc tính	Ý nghĩa	Vai trò trong reconstruction
Timestamp	Thời gian ghi nhận log của xe	Một phần của khóa ghép
VIN	Mã định danh phương tiện	Một phần của khóa ghép và dùng để chia horizontal fragment
Speed	Tốc độ hiện tại của xe	Thuộc nhóm Operational
FuelLevel	Mức nhiên liệu hiện tại	Thuộc nhóm Operational
Location	Vị trí hiện tại của xe	Thuộc nhóm Operational
EngineTemp	Nhiệt độ động cơ	Thuộc nhóm Diagnostic

Khóa định danh một bản ghi là VIN + Timestamp. Lý do sử dụng khóa ghép là vì một xe có thể xuất hiện nhiều lần tại nhiều thời điểm khác nhau. Nếu chỉ dùng VIN, quá trình join các fragment dọc có thể ghép sai hoặc tạo ra bản ghi không chính xác.

1.3. Fragmentation Strategy

1.3.1. Horizontal Fragmentation

Dataset được chia thành 4 horizontal fragments dựa trên khoảng giá trị VIN. Mỗi fragment ngang chứa một tập con các dòng của dataset gốc nhưng vẫn giữ đầy đủ các thuộc tính ban đầu.

Fragment	Khoảng VIN	Node lưu trữ
H1	VIN000001 - VIN002500	Node A
H2	VIN002501 - VIN005000	Node B
H3	VIN005001 - VIN007500	Node C
H4	VIN007501 - VIN010000	Node D

Yêu cầu khôi phục đối với phân mảnh ngang: $H1 \cup H2 \cup H3 \cup H4 = \text{VehicleLogs.csv}$. Đây là cơ sở để kiểm tra lossless union.

1.3.2. Vertical Fragmentation

Mỗi horizontal fragment tiếp tục được chia thành 2 vertical fragments. Cả hai fragment dọc đều giữ lại VIN và Timestamp để bảo đảm có thể join chính xác.

Loại fragment	Các thuộc tính
Operational	Timestamp, VIN, Speed, FuelLevel, Location
Diagnostic	Timestamp, VIN, EngineTemp

Yêu cầu khôi phục đối với phân mảnh dọc: $\text{Operational} \bowtie \text{Diagnostic} = \text{Fragment ngang ban đầu}$. Đây là cơ sở để kiểm tra lossless join.

Sau khi phân mảnh, hệ thống sẽ tạo ra 8 file: H1_Operational.csv, H1_Diagnostic.csv, H2_Operational.csv, H2_Diagnostic.csv, H3_Operational.csv, H3_Diagnostic.csv, H4_Operational.csv và H4_Diagnostic.csv.

CHƯƠNG III: KIẾN TRÚC HỆ THỐNG

I. Nodes

Hệ thống mô phỏng 4 site phân tán, tương ứng với 4 horizontal fragments. Mỗi node lưu trữ 2 file CSV đại diện cho 2 vertical fragments.

Node	Fragment lưu trữ	Ý nghĩa
Node A	H1_Operational.csv, H1_Diagnostic.csv	Dữ liệu xe có VIN trong khoảng H1
Node B	H2_Operational.csv, H2_Diagnostic.csv	Dữ liệu xe có VIN trong khoảng H2
Node C	H3_Operational.csv, H3_Diagnostic.csv	Dữ liệu xe có VIN trong khoảng H3
Node D	H4_Operational.csv, H4_Diagnostic.csv	Dữ liệu xe có VIN trong khoảng H4

II. Communication Layer

Phiên bản chính của project sử dụng mô phỏng phân tán bằng thư mục cục bộ. Mỗi node được biểu diễn bằng một folder riêng. Reconstruction script đọc dữ liệu từ các folder node này để tái dựng dataset. Cách làm này phù hợp với thời gian triển khai ngắn, dễ demo và tập trung trực tiếp vào bản chất của phân mảnh và khôi phục dữ liệu.

Nếu có thời gian mở rộng, nhóm có thể bổ sung Flask REST API để mỗi node cung cấp fragment thông qua endpoint HTTP. Tuy nhiên, phần bắt buộc của đề tài sẽ ưu tiên Python, CSV và node-folder simulation.

III. Storage

Thư mục/File	Mục đích
data/VehicleLogs.csv	Lưu dataset gốc 10.000 dòng
nodes/node_A/	Lưu fragment H1
nodes/node_B/	Lưu fragment H2
nodes/node_C/	Lưu fragment H3
nodes/node_D/	Lưu fragment H4
outputs/reconstructed.csv	Lưu dataset sau khi khôi phục
outputs/validation_report.csv	Lưu báo cáo missing/corrupted records

CHƯƠNG IV: NỀN TẢNG CÔNG NGHỆ VÀ KẾ HOẠCH THỰC HIỆN

Thành phần	Công nghệ sử dụng	Lý do chọn
Programming Language	Python	Dễ xử lý file CSV, phù hợp với data processing và demo nhanh
Data Processing	pandas	Hỗ trợ đọc/ghi CSV, join, union, compare dữ liệu
Storage	CSV files	Đơn giản, dễ kiểm tra trực tiếp và phù hợp với yêu cầu dataset
Deployment	Localhost folder-based simulation	Mô phỏng node phân tán bằng thư mục, giảm độ phức tạp triển khai
Version Control	GitHub/GitLab	Lưu trữ code và README hướng dẫn chạy
Optional Extension	Flask REST API hoặc Docker	Mở rộng nếu cần mô phỏng giao tiếp mạng hoặc node độc lập

I. Cấu trúc project dự kiến

Cấu trúc thư mục dự kiến của repository như sau:

```
vehicle-telematics-validator/  
├── data/  
│   └── VehicleLogs.csv  
├── nodes/  
│   ├── node_A/  
│   ├── node_B/  
│   ├── node_C/  
│   └── node_D/  
├── outputs/  
│   ├── reconstructed.csv  
│   └── validation_report.csv  
└── src/  
    ├── generate_dataset.py  
    ├── fragment_data.py  
    ├── inject_failure.py  
    └── reconstruct_validate.py
```

└─ README.md
└─ requirements.txt

II. Các script chính

Script	Chức năng
generate_dataset.py	Tạo VehicleLogs.csv gồm 10.000 dòng dữ liệu giả lập.
fragment_data.py	Chia dataset thành 4 fragment ngang và 8 fragment dọc.
inject_failure.py	Xóa 5 dòng ngẫu nhiên từ một fragment để mô phỏng lỗi.
reconstruct_validate.py	Khôi phục dữ liệu và xuất báo cáo validation_report.csv.

CHƯƠNG V: TIÊU CHÍ THÀNH CÔNG VÀ ĐÁNH GIÁ

I. Quantitative Metrics

Metric	Cách đo	Kết quả mong đợi
Original record count	Đếm số dòng trong VehicleLogs.csv	10.000 records
Reconstructed record count	Đếm số dòng sau reconstruction	10.000 trước lỗi; có thể thiếu sau khi inject failure
Missing records	So sánh khóa VIN + Timestamp giữa original và reconstructed	0 trước lỗi, 5 sau lỗi
Corrupted records	So sánh giá trị từng cột giữa original và reconstructed	0 nếu chỉ mô phỏng mất dòng
Duplicate records	Kiểm tra trùng khóa VIN + Timestamp	0
Execution time	Đo thời gian chạy reconstruction_validate.py	Báo cáo theo milliseconds

II. Failure Scenario

Kịch bản lỗi chính của đề tài là xóa 5 dòng ngẫu nhiên từ file H2_Diagnostic.csv tại Node B. Đây là mô phỏng trường hợp một fragment ở site phân tán bị mất một phần dữ liệu. Khi chạy validator, hệ thống phải phát hiện chính xác 5 bản ghi bị ảnh hưởng.

Output mong đợi trong `validation_report.csv` gồm các trường: VIN, Timestamp, ErrorType, MissingFragment, MissingColumn và Description. Ví dụ: ErrorType = Missing Diagnostic Record, MissingFragment = H2_Diagnostic.csv, MissingColumn = EngineTemp.

CHƯƠNG VI: KẾT LUẬN

Project Vehicle Telematics Reconstruction Validator tập trung vào một vấn đề cốt lõi của cơ sở dữ liệu phân tán: làm thế nào để đảm bảo dữ liệu sau khi phân mảnh vẫn có thể được khôi phục chính xác. Thông qua việc chia dataset thành các fragment ngang và dọc, sau đó mô phỏng lỗi mất dữ liệu trong một fragment, hệ thống sẽ chứng minh khả năng kiểm tra lossless union, lossless join và reconstruction correctness.

Với phạm vi triển khai bằng Python, pandas và CSV, đề tài có tính khả thi cao trong thời gian ngắn, dễ demo và phù hợp với yêu cầu đánh giá cuối môn. Nếu có thêm thời gian, hệ thống có thể được mở rộng bằng cách triển khai các node độc lập thông qua Flask REST API hoặc Docker containers.